

◆ HavenWorld ◆

Complete Build Guide

Part 4A of 4 — Phase 1 Linux: Server Bootstrap, Prisma Schema, and Database

Developer:	Daniel Boone
Repository:	github.com/dboone323/HavenWorld
Game Server:	https://[your-oracle-ip].nip.io
Date:	September 18, 2026
Series Part:	4A of 4 (Part 4, Sub-Part A)
Scope:	Oracle Cloud ARM VM via SSH from MacBook Pro

HavenWorld Build Guide Series

Part 1

Project Setup & Monorepo Foundation

Part 2

Client Build — Phaser, UI, Asset Pipeline

Part 3

Infrastructure — Oracle VM, Nginx, PostgreSQL, Redis, PM2

Part 4A ► YOU ARE HERE

Phase 1 Linux: Server Bootstrap, Prisma Schema, Database

Part 4B

Auth Routes, Socket.io Handlers, RoomManager, ModerationService

Part 4C

Admin Routes, Invite Codes, Deployment Workflow, Alpha-Ready Checklist

Introduction

This document covers the first half of Phase 1 Linux work for HavenWorld. All steps tagged [LINUX] are performed on the Oracle Cloud ARM VM via SSH from the MacBook Pro. Steps tagged [MAC] are authored on the Mac and committed/pushed before the VM pulls. The Mac developer writes and pushes code; the Oracle VM pulls and runs it. By the end of Part 4A, the Oracle VM will be in the following state:

- A fully initialized Node.js + Express + Socket.io server with a working /health endpoint
- A complete Prisma database schema covering all **10 tables**
- All migrations applied to the havenworld_prod PostgreSQL database
- All seed data inserted (public rooms, default free items, personal room template)
- The server running under PM2 and accessible at [https://\[ip\].nip.io/health](https://[ip].nip.io/health)

> Scope of This Document

Part 4B covers the auth system, Socket.io handlers, RoomManager, and ModerationService. **Part 4C** covers admin routes, invite codes, the deployment workflow, and the final alpha-ready checklist. Each sub-part builds directly on the previous — do not skip ahead.

Tag Legend

Tag	Meaning	Action Required
[LINUX]	Runs on Oracle Cloud ARM VM	SSH into VM, then execute the commands shown
[MAC]	Runs on MacBook Pro (local dev machine)	Create/edit files locally, commit, and push to main
[MAC then LINUX]	Action split across both machines	Mac portion first, then VM portion after pull

6 Server Bootstrap [LINUX]

6.1 Server Entry Point

This section establishes the core Node.js server infrastructure. Steps proceed in dependency order: install packages first, configure TypeScript, then wire up the entry point. The Mac handles all authoring; the Oracle VM handles execution after a `git pull`.

[LINUX-1-A1] Install Server Dependencies [MAC then LINUX]

The Mac developer runs this first locally, commits `package.json` and `package-lock`, then the Oracle VM installs from the lockfile.

On Mac, from `apps/server/`:

```
pnpm add express socket.io @prisma/client bcryptjs jsonwebtoken pnpm add
cookie-parser cors helmet morgan express-rate-limit pnpm add bad-words
@sentry/node resend zod redis pnpm add -D typescript ts-node nodemon jest ts-
jest pnpm add -D @types/node @types/express @types/bcryptjs pnpm add -D
@types/jsonwebtoken @types/cookie-parser @types/morgan pnpm add -D eslint
```

@typescript-eslint/parser @typescript-eslint/eslint-plugin

Commit and push to develop, merge to main.

On Oracle VM:

```
cd /opt/havenworld git pull origin main pnpm install --frozen-lockfile node --version  
# → v22.x.x (confirm Node is running)
```

LINUX-1-A2 Create apps/server/package.json Scripts Section [MAC]

Ensure the scripts section of apps/server/package.json contains the following entries. The prisma.seed key registers the seed script so that npx prisma db seed finds it automatically.

```
"scripts": {  "dev": "nodemon --watch src --ext ts --exec ts-node src/index.ts",  
  "build": "tsc",  "start": "node dist/index.js",  "test": "jest --passWithNoTests --  
  forceExit",  "lint": "eslint src --ext .ts --max-warnings 0" },  "prisma": {  "seed": "ts-  
  node prisma/seed.ts" }
```

LINUX-1-A3 Create apps/server/tsconfig.json [MAC]

Create this file on Mac and commit. The paths alias maps @shared/* to the monorepo shared package, enabling type-safe imports across the workspace.

```
{  "compilerOptions": {    "target": "ES2022",    "module": "commonjs",    "lib": ["ES2022"],    "outDir": "./dist",    "rootDir": "./src",    "strict": true,  
    "esModuleInterop": true,    "resolveJsonModule": true,    "skipLibCheck": true,  
    "declaration": true,    "declarationMap": true,    "sourceMap": true,    "paths":  
    {      "@shared/*": ["../../packages/shared/src/*"]    }  },  "include": ["src/**/*",  
    "prisma/seed.ts"],  "exclude": ["node_modules", "dist", "**/*.test.ts"] }
```

LINUX-1-A4 Create the Redis Client Singleton (src/redis.ts) [MAC]

Create apps/server/src/redis.ts. This singleton pattern ensures only one Redis connection is maintained across the entire process lifetime. The connectRedis() function is called once during startup in index.ts.

```
import { createClient } from 'redis'; const redisUrl = process.env.REDIS_URL; if (!
redisUrl) throw new Error('REDIS_URL environment variable is not set'); export
const redis = createClient({ url: redisUrl }); redis.on('error', (err) =>
console.error('[Redis] Client error:', err)); redis.on('connect', () =>
console.log('[Redis] Connected successfully')); redis.on('reconnecting', () =>
console.warn('[Redis] Reconnecting...')); export const connectRedis = async ():
Promise<void> => { await redis.connect(); console.log('[Redis] Ready'); };
```

LINUX-1-A5 Create the Prisma Client Singleton (src/prisma.ts) [MAC]

Create apps/server/src/prisma.ts. The global guard prevents multiple Prisma client instances from being created during hot-reload in development, which would exhaust the PostgreSQL connection pool.

```
import { PrismaClient } from '@prisma/client'; const globalForPrisma = global as
unknown as { prisma: PrismaClient }; export const prisma =
globalForPrisma.prisma ?? new PrismaClient({ log:
process.env.NODE_ENV === 'development' ? ['query', 'info', 'warn', 'error']
: ['warn', 'error'], }); if (process.env.NODE_ENV !== 'production')
{ globalForPrisma.prisma = prisma; } export default prisma;
```

LINUX-1-A6 Create the Server Entry Point (src/index.ts) [MAC]

Create `apps/server/src/index.ts` — the main application file. This file wires together Express, Socket.io, all middleware, the health endpoint, all API route mounts, Sentry error handling, and graceful shutdown. Route files (`auth`, `users`, etc.) are stubs for now and will be populated in Part 4B and Part 4C.

```
import 'dotenv/config'; import express from 'express'; import { createServer } from
'http'; import { Server } from 'socket.io'; import cors from 'cors'; import helmet from
'helmet'; import cookieParser from 'cookie-parser'; import morgan from 'morgan';
import * as Sentry from '@sentry/node'; import { connectRedis } from './redis';
import { prisma } from './prisma'; // — Sentry (error monitoring)
```

```
Sentry.init({ dsn: process.env.SENTRY_DSN, environment:
process.env.NODE_ENV ?? 'development', tracesSampleRate: 0.1, }); // —
Express app
```

```
————— const app = express(); const httpServer = createServer(app); // —
Socket.io
```

```
————— const io = new Server(httpServer, { cors: { origin:
process.env.CLIENT_URL, credentials: true, }, transports: ['websocket',
'polling'], pingTimeout: 60000, pingInterval: 25000, }); // — Middleware
```

```
————— app.use(Sentry.Handlers.requestHandler());
app.use(helmet({ contentSecurityPolicy: false, // Phaser handles its own CSP
needs crossOriginEmbedderPolicy: false, })); app.use(cors({ origin:
process.env.CLIENT_URL, credentials: true, methods: ['GET', 'POST', 'PUT',
'PATCH', 'DELETE', 'OPTIONS'], })); app.use(express.json({ limit: '10kb' }));
app.use(express.urlencoded({ extended: false, limit: '10kb' }));
app.use(cookieParser()); if (process.env.NODE_ENV !== 'test')
```

```
{ app.use(morgan('combined')); } // — Health check
```

```
————— // UptimeRobot and GitHub Actions health checks hit this endpoint
app.get('/health', async (_req, res) => { try { await prisma.$queryRaw`SELECT
1`; // verify DB is reachable res.json({ status: 'ok', timestamp: new
Date().toISOString(), version: process.env.npm_package_version ?? '0.1.0',
uptime: Math.floor(process.uptime()), }); } catch
{ res.status(503).json({ status: 'error', message: 'Database
unreachable' }); }); // — API Routes
```

```
————— import authRoutes from './routes/auth'; import userRoutes from
'./routes/users'; import roomRoutes from './routes/rooms'; import friendRoutes from
'./routes/friends'; import reportRoutes from './routes/reports'; import adminRoutes
from './routes/admin'; app.use('/api/auth', authRoutes); app.use('/api/users',
userRoutes); app.use('/api/rooms', roomRoutes); app.use('/api/friends',
friendRoutes); app.use('/api/reports', reportRoutes); app.use('/api/admin',
adminRoutes); // — 404 catch-all
```

```
————— app.use((_req, res) => { res.status(404).json({ error: 'Not found' }); }); //
— Sentry error handler
```

```
app.use(Sentry.Handlers.errorHandler()); // — Global error handler
```

```
app.use((err: Error, _req: express.Request, res: express.Response, _next:
express.NextFunction) => { console.error('[Error]', err.message);
res.status(500).json({ error: 'Internal server error' }); }); // — Socket.io handlers
```

```
— import { registerSocketHandlers } from './sockets'; registerSocketHandlers(io); //
```

— Startup

```
const PORT = parseInt(process.env.PORT ?? '3000', 10);
async function start() { try { await connectRedis(); await prisma.$connect();
console.log('[DB] PostgreSQL connected'); httpServer.listen(PORT, '0.0.0.0', ()
=> { console.log('[Server] HavenWorld listening on port ${PORT}');
console.log('[Server] Environment: ${process.env.NODE_ENV}');
console.log('[Server] Client origin: ${process.env.CLIENT_URL}'); }); } catch
(err) { console.error('[Startup] Failed to start server:', err);
process.exit(1); }} // — Graceful shutdown
```

```
— process.on('SIGTERM', async () => { console.log('[Server] SIGTERM received
— shutting down gracefully'); httpServer.close(async () => { await prisma.
$disconnect(); console.log('[Server] Shutdown complete');
process.exit(0); }); }); start(); export { io };
```

LINUX-1-A7 Verify the Server Compiles and Runs **[LINUX]**

On Oracle VM after Mac pushes and VM pulls:

```
cd /opt/havenworld/apps/server pnpm build # Should print: no errors — dist/
directory appears node dist/index.js # Should print: # [Redis] Connected
successfully # [Redis] Ready # [DB] PostgreSQL connected # [Server]
HavenWorld listening on port 3000
```

Test the health endpoint in a second terminal session:

```
curl http://localhost:3000/health
```

Expected response:

```
{"status":"ok","timestamp":"2026-09-18T...","version":"0.1.0","uptime":4}
```

Press `ctrl+c` to stop the process. PM2 will manage the process from Part 4C onward.

✓ Milestone: Section 6 Complete

At this point the server compiles cleanly, connects to both PostgreSQL and Redis, and responds with `{"status":"ok"}` at `/health`. This is your baseline. Every subsequent section builds on this running server.

7 Prisma Schema and Database [LINUX]

7.1 Initialize Prisma

LINUX-1-B1 Initialize Prisma in `apps/server` [MAC]

On Mac, from `apps/server/`:

```
npx prisma init --datasource-provider postgresql
```

This creates `prisma/schema.prisma` and updates `.env` with a `DATABASE_URL` placeholder. Open `prisma/schema.prisma` — it currently has a minimal generator and datasource block. The following steps replace its contents with the full HavenWorld schema.

LINUX-1-B2 Configure the Prisma Generator and Datasource [MAC]

Replace the top of `prisma/schema.prisma` with the following. The `fullTextSearch` preview feature enables Prisma's PostgreSQL full-text search operators, which are used for username lookup in the friend search route (Part 4C).

```
generator client {
  provider      = "prisma-client-js"
  previewFeatures = ["fullTextSearch"]
}
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

7.2 Full Database Schema

The HavenWorld schema consists of **10 tables** and **7 enums**. All models use UUID primary keys for portability and security. The steps below define the models in dependency order — referenced models must be defined before the models that reference them.

Table	Model	Primary Relations	Step
users	User	Root — referenced by all other models	LINUX-1-B3
avatars	Avatar	1-to-1 with User	LINUX-1-B4
items	Item	Referenced by Inventory, RoomFurniture	LINUX-1-B5
inventories	Inventory	Many-to-many junction: User + Item	LINUX-1-B5
rooms	Room	Optional owner (User), contains RoomFurniture + ChatMessage	LINUX-1-B6
room_furniture	RoomFurniture	Belongs to Room + Item, placed by User	LINUX-1-B6
chat_messages	ChatMessage	Belongs to Room (optional), sent by User	LINUX-1-B7
friends	Friend	Self-referential User pair with status	LINUX-1-B7
reports	Report	Reporter User +	LINUX-1-B7

Table	Model	Primary Relations	Step
		Reported User	
invite_codes	InviteCode	Creator User + optional UsedBy User	LINUX-1-B7

LINUX-1-B3 Define the Users Model [MAC]

Add to `prisma/schema.prisma`. The `Role` enum controls feature access. The `UserStatus` enum drives the moderation system (muted players have a `mutedUntil` timestamp; banned players cannot log in at all).

```
enum Role { PLAYER MODERATOR ADMIN } enum UserStatus { ACTIVE
MUTED BANNED } model User { id          String    @id @default(uuid())
username      String    @unique email          String    @unique
passwordHash   String   role              Role      @default(PLAYER) status
UserStatus @default(ACTIVE) emailVerified   Boolean    @default(false)
emailVerifyToken String? mutedUntil        DateTime? createdAt      DateTime
@default(now()) lastLoginAt      DateTime? avatar          Avatar? inventory
Inventory[] ownedRooms           Room[]      @relation("RoomOwner")
placedFurniture RoomFurniture[] sentMessages ChatMessage[]
sentReports     Report[]        @relation("Reporter") receivedReports Report[]
@relation("Reported") initiatedFriends Friend[]    @relation("Requester")
receivedFriends Friend[]        @relation("Addressee") createdInviteCodes
InviteCode[] @relation("InviteCreator") usedInviteCode InviteCode?
@relation("InviteUser") @@map("users") }
```

LINUX-1-B4 Define the Avatars Model [MAC]

Add to `prisma/schema.prisma`. The Avatar has **9 outfit slots** (head, face, body, legs, feet, back, hand, plus `hairStyle` and `eyeStyle`). Nullable outfit slots mean a

slot can be empty — the avatar renderer falls back to the base skin. The `onDelete: Cascade` ensures the avatar row is deleted when the owning user is deleted.

```
model Avatar {
  id          String @id @default(uuid())
  userId      String @unique
  user        User   @relation(fields: [userId], references: [id], onDelete: Cascade)
  bodyType    String @default("default")
  skinTone    String @default("light")
  hairStyle   String @default("hair-short-01")
  hairColor   String @default("brown")
  eyeStyle    String @default("eyes-default")
  eyeColor    String @default("brown")
  outfitHead  String?
  outfitFace  String?
  outfitBody  String?
  outfitLegs  String?
  outfitFeet  String?
  outfitBack  String?
  outfitHand  String?
  updatedAt   DateTime @updatedAt
  @@map("avatars")
}
```

LINUX-1-B5 Define Items and Inventory Models [MAC]

Add to `prisma/schema.prisma`. The `@@unique([userId, itemId])` constraint on `Inventory` ensures a player cannot hold duplicate entries of the same item — quantity is tracked by the `quantity` field instead. The `skipDuplicates` flag in `createMany()` calls relies on this constraint.

```
enum ItemCategory {
  HAIR
  EYES
  CLOTHING_HEAD
  CLOTHING_FACE
  CLOTHING_BODY
  CLOTHING_LEGS
  CLOTHING_FEET
  CLOTHING_BACK
  CLOTHING_HAND
  FURNITURE
  DECORATION
}
enum ItemRarity {
  COMMON
  UNCOMMON
  RARE
  EPIC
  LEGENDARY
}
model Item {
  id          String @id @default(uuid())
  name        String
  description  String @default("")
  category    ItemCategory
  rarity       ItemRarity @default(COMMON)
  isTradeable Boolean @default(false)
  spriteKey   String
  createdAt   DateTime @default(now())
  inventories Inventory[]
  roomFurniture RoomFurniture[]
  @@map("items")
}
model Inventory {
  id          String @id @default(uuid())
  userId      String
  user        User   @relation(fields: [userId],
```

```

references: [id], onDelete: Cascade) itemId String item Item
@relation(fields: [itemId], references: [id]) quantity Int @default(1)
acquiredAt DateTime @default(now()) isEquipped Boolean @default(false)
@@unique([userId, itemId]) @@map("inventories") }

```

LINUX-1-B6 Define Rooms and Room Furniture Models [MAC]

Add to `prisma/schema.prisma`. The `ownerId` is nullable so that public system rooms (Lobby, Town Square, etc.) can exist without an owner. The `onDelete: SetNull` on the owner relation means public rooms persist if their creator account is deleted. `RoomFurniture` stores the `x`, `y`, `z`, `rotation`, and `layer` required by the Phaser isometric renderer.

```

model Room { id String @id @default(uuid()) name String
description String @default("") ownerId String? owner User?
@relation("RoomOwner", fields: [ownerId], references: [id], onDelete: SetNull)
isPublic Boolean @default(false) maxOccupants Int @default(50) theme
String @default("default") width Int @default(20) height Int
@default(15) backgroundKey String @default("map-lobby") createdAt
DateTime @default(now()) updatedAt DateTime @updatedAt furniture
RoomFurniture[] chatMessages ChatMessage[] @@map("rooms") } model
RoomFurniture { id String @id @default(uuid()) roomId String room
Room @relation(fields: [roomId], references: [id], onDelete: Cascade) itemId
String item Item @relation(fields: [itemId], references: [id]) placedBy String
placer User @relation(fields: [placedBy], references: [id]) x Int y Int
z Int @default(0) rotation Int @default(0) layer Int @default(0)
placedAt DateTime @default(now()) @@map("room_furniture") }

```

LINUX-1-B7 Define Chat, Friends, Reports, and InviteCodes Models [MAC]

Add to prisma/schema.prisma. The `@@index([roomId, createdAt])` on `ChatMessage` makes room-scoped chat history queries efficient. The `@@unique([requesterId, addresseeId])` on `Friend` prevents duplicate friend requests. The `@@index([code, isActive])` on `InviteCode` supports fast code lookups at registration.

```
model ChatMessage {
  id          String @id @default(uuid())
  roomId      String?
  room        Room? @relation(fields: [roomId], references: [id], onDelete: SetNull)
  senderId    String
  sender      User @relation(fields: [senderId], references: [id], onDelete: Cascade)
  content     String
  isFiltered  Boolean @default(false)
  createdAt   DateTime @default(now()) @@index([roomId, createdAt])
  @@map("chat_messages")
}
enum FriendStatus {
  PENDING
  ACCEPTED
  BLOCKED
}
model Friend {
  id          String @id @default(uuid())
  requesterId String
  requester   User @relation("Requester", fields: [requesterId], references: [id], onDelete: Cascade)
  addresseeId String
  addressee   User @relation("Addressee", fields: [addresseeId], references: [id], onDelete: Cascade)
  status      FriendStatus @default(PENDING)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  @@unique([requesterId, addresseeId])
  @@map("friends")
}
enum ReportReason {
  HARASSMENT
  SPAM
  INAPPROPRIATE_CONTENT
  IMPERSONATION
  CHEATING
  OTHER
}
enum ReportStatus {
  OPEN
  REVIEWED
  RESOLVED
  DISMISSED
}
model Report {
  id          String @id @default(uuid())
  reporterId  String
  reporter    User @relation("Reporter", fields: [reporterId], references: [id], onDelete: Cascade)
  reportedUserId String
  reportedUser User @relation("Reported", fields: [reportedUserId], references: [id], onDelete: Cascade)
  reason      ReportReason
  description  String @default("")
  status      ReportStatus @default(OPEN)
  moderatorNotes String?
  createdAt   DateTime @default(now())
  resolvedAt  DateTime?
  @@index([status, createdAt])
  @@map("reports")
}
model InviteCode {
  id          String @id @default(uuid())
```

```
code String @unique createdBy String creator User
@relation("InviteCreator", fields: [createdBy], references: [id]) usedById String?
@unique usedBy User? @relation("InviteUser", fields: [usedById], references:
[id]) usedAt DateTime? expiresAt DateTime isActive Boolean @default(true)
createdAt DateTime @default(now()) @@index([code, isActive])
@@map("invite_codes") }
```

7.3 Running Migrations

⚠ Critical: Never Run `migrate dev` on Production

Use `npx prisma migrate deploy` on the Oracle VM at all times. The `migrate dev` command is only for your local Mac environment — it resets data and creates shadow databases. Running it on the production VM will destroy data in `havenworld_prod`.

LINUX-1-B8 Run the Initial Migration on the Oracle VM **[LINUX]**

After Mac commits the full schema and pushes to main, on the Oracle VM:

```
cd /opt/havenworld git pull origin main cd apps/server npx prisma migrate deploy
```

Verify tables were created:

```
psql -U havenworld -d havenworld_prod -h 127.0.0.1 -c "\dt"
```

Expected output — list of **10 tables**:

List of relations	Schema	Name	Type	Owner
public	avatars	table	havenworld	public
public	chat_messages	table	havenworld	public
public	friends	table	havenworld	public
public	inventories	table	havenworld	public

```
| table | havenworld public | invite_codes | table | havenworld public | items
| table | havenworld public | reports | table | havenworld public |
room_furniture | table | havenworld public | rooms | table | havenworld
public | users | table | havenworld
```

› Dev Reset (Mac Only)

If you ever need to reset the schema on your Mac dev environment (never on production): `npx prisma migrate reset`. This **deletes all local data** and re-applies migrations from scratch.

LINUX-1-B9 Generate the Prisma Client After Schema Changes **[LINUX]**

Whenever the schema changes, regenerate the type-safe client. This is also run automatically when `pnpm build` executes, because the build script triggers the Prisma generate hook.

```
cd /opt/havenworld/apps/server npx prisma generate
```

Expected output: Prisma Client generated into `node_modules/.prisma/client` with no errors.

7.4 Database Seeding

LINUX-1-B10 Create the Seed Script (`prisma/seed.ts`) **[MAC]**

Create `apps/server/prisma/seed.ts`. This inserts **4 public rooms** and **17 default free items** using `upsert()` so the script is idempotent — running it multiple times does not create duplicates. Fixed UUIDs (e.g. `'room-lobby'`)

make the rooms stable and referenceable from application code.

```
import { PrismaClient, ItemCategory, ItemRarity } from '@prisma/client'; const
prisma = new PrismaClient(); async function main() { console.log('[Seed] Starting
database seed...'); // — 1. Public Rooms
```

```
— console.log('[Seed] Creating public rooms...'); const publicRooms =
[ { id: 'room-lobby', name: 'The Lobby', description: 'The
main gathering place for all HavenWorld visitors.', isPublic: true,
maxOccupants: 50, width: 20, height: 15, backgroundKey:
'map-lobby', theme: 'lobby', }, { id: 'room-town-square',
name: 'Town Square', description: 'The heart of HavenWorld — meet
friends and explore.', isPublic: true, maxOccupants: 100, width:
30, height: 20, backgroundKey: 'map-town-square', theme:
'outdoor', }, { id: 'room-park', name: 'Haven Park',
description: 'A peaceful outdoor space to relax and socialize.', isPublic:
true, maxOccupants: 50, width: 25, height: 18,
backgroundKey: 'map-park', theme: 'outdoor', }, { id: 'room-
cafe', name: 'The Cozy Café', description: 'Grab a virtual coffee and
catch up with friends.', isPublic: true, maxOccupants: 30, width:
15, height: 12, backgroundKey: 'map-cafe', theme:
'indoor', }, ]; for (const room of publicRooms) { await prisma.room.upsert({
where: { id: room.id }, create: room, update: { name: room.name,
description: room.description }, }); } console.log(`[Seed] Created $
{publicRooms.length} public rooms`); // — 2. Default Free Items
```

```
console.log('[Seed] Creating default free items...'); const defaultItems = [ // Hair
{ id: 'hair-short-01', name: 'Short Hair', category: ItemCategory.HAIR,
spriteKey: 'hair/hair-short-01', rarity: ItemRarity.COMMON }, { id: 'hair-short-02',
```

```
name: 'Short Alt Hair', category: ItemCategory.HAIR,      spriteKey: 'hair/hair-
short-02',  rarity: ItemRarity.COMMON },  { id: 'hair-long-01',  name: 'Long Hair',
category: ItemCategory.HAIR,      spriteKey: 'hair/hair-long-01',  rarity:
ItemRarity.COMMON },  // Eyes  { id: 'eyes-default',  name: 'Default Eyes',
category: ItemCategory.EYES,      spriteKey: 'eyes/eyes-default',  rarity:
ItemRarity.COMMON },  { id: 'eyes-round',  name: 'Round Eyes',  category:
ItemCategory.EYES,      spriteKey: 'eyes/eyes-round',  rarity:
ItemRarity.COMMON },  // Tops  { id: 'shirt-white',  name: 'White Shirt',
category: ItemCategory.CLOTHING_BODY, spriteKey: 'clothing/shirt-white', rarity:
ItemRarity.COMMON },  { id: 'shirt-black',  name: 'Black Shirt',  category:
ItemCategory.CLOTHING_BODY, spriteKey: 'clothing/shirt-black', rarity:
ItemRarity.COMMON },  { id: 'shirt-blue',  name: 'Blue Shirt',  category:
ItemCategory.CLOTHING_BODY, spriteKey: 'clothing/shirt-blue', rarity:
ItemRarity.COMMON },  // Bottoms  { id: 'pants-blue',  name: 'Blue Jeans',
category: ItemCategory.CLOTHING_LEGS, spriteKey: 'clothing/pants-blue', rarity:
ItemRarity.COMMON },  { id: 'pants-black',  name: 'Black Pants',  category:
ItemCategory.CLOTHING_LEGS, spriteKey: 'clothing/pants-black', rarity:
ItemRarity.COMMON },  // Feet  { id: 'shoes-white',  name: 'White Sneakers',
category: ItemCategory.CLOTHING_FEET, spriteKey: 'clothing/shoes-white',
rarity: ItemRarity.COMMON },  { id: 'shoes-black',  name: 'Black Shoes',
category: ItemCategory.CLOTHING_FEET, spriteKey: 'clothing/shoes-black',
rarity: ItemRarity.COMMON },  // Furniture (starter set for personal rooms)  { id:
'furniture-chair', name: 'Basic Chair',  category: ItemCategory.FURNITURE,
spriteKey: 'furniture/chair-basic', rarity: ItemRarity.COMMON },  { id: 'furniture-
table', name: 'Basic Table',  category: ItemCategory.FURNITURE,  spriteKey:
'furniture/table-basic', rarity: ItemRarity.COMMON },  { id: 'furniture-plant', name:
'Small Plant',  category: ItemCategory.FURNITURE,  spriteKey: 'furniture/plant-
small', rarity: ItemRarity.COMMON },  { id: 'furniture-rug',  name: 'Basic Rug',
```

```
category: ItemCategory.FURNITURE, spriteKey: 'furniture/rug-basic', rarity:
ItemRarity.COMMON }, { id: 'furniture-lamp', name: 'Floor Lamp', category:
ItemCategory.FURNITURE, spriteKey: 'furniture/lamp-floor', rarity:
ItemRarity.COMMON }, ]; for (const item of defaultItems) { await
prisma.item.upsert({ where: { id: item.id }, create: { ...item, isTradeable:
false }, update: { name: item.name }, }); } console.log(`[Seed] Created $
{defaultItems.length} default items`); console.log('[Seed] Database seed
complete!'); } main() .catch((err) => { console.error('[Seed] Error:', err);
process.exit(1); }) .finally(() => prisma.$disconnect());
```

LINUX-1-B11 Run the Seed Script on Oracle VM [LINUX]

After Mac pushes the seed file:

```
cd /opt/havenworld git pull origin main cd apps/server npx prisma db seed
```

Expected output:

```
[Seed] Starting database seed... [Seed] Creating public rooms... [Seed] Created 4
public rooms [Seed] Creating default free items... [Seed] Created 17 default items
[Seed] Database seed complete!
```

Verify the seed data in PostgreSQL:

```
psql -U havenworld -d havenworld_prod -h 127.0.0.1 SELECT id, name, "isPublic"
FROM rooms; SELECT id, name, category FROM items ORDER BY category; \q
```

LINUX-1-B12 Create the Inventory Grant Helper

(src/services/InventoryService.ts) [MAC]

This service is called at registration to automatically grant all 17 default free

items to new players. `createMany` with `skipDuplicates: true` makes the grant call safe to call multiple times without throwing on re-registration edge cases.

```
import { prisma } from '../prisma'; const DEFAULT_FREE_ITEM_IDS = [ 'hair-short-01', 'hair-short-02', 'hair-long-01', 'eyes-default', 'eyes-round', 'shirt-white', 'shirt-black', 'shirt-blue', 'pants-blue', 'pants-black', 'shoes-white', 'shoes-black', 'furniture-chair', 'furniture-table', 'furniture-plant', 'furniture-rug', 'furniture-lamp', ]; export const inventoryService = { async grantDefaultItems(userId: string): Promise<void> { await prisma.inventory.createMany({ data: DEFAULT_FREE_ITEM_IDS.map((itemId) => ({ userId, itemId })), skipDuplicates: true, }); }, async getUserInventory(userId: string) { return prisma.inventory.findMany({ where: { userId }, include: { item: true }, orderBy: [{ item: { category: 'asc' } }, { acquiredAt: 'desc' }], }); }, async hasItem(userId: string, itemId: string): Promise<boolean> { const entry = await prisma.inventory.findUnique({ where: { userId_itemId: { userId, itemId } }, }); return !!entry; }, async hasAllItems(userId: string, itemIds: string[]): Promise<boolean> { const count = await prisma.inventory.count({ where: { userId_itemId: { in: itemIds } }, }); return count === itemIds.length; }, };
```

7.5 Auth Middleware

LINUX-1-B13 Create `src/middleware/auth.ts` **[MAC]**

This middleware is used by all protected API routes and the Socket.io connection. `requireAuth` validates the Bearer JWT and attaches the decoded payload to `req.user`. `requireRole` is a factory that returns a middleware enforcing the caller belongs to one of the specified roles. Error codes (`NO_TOKEN`, `TOKEN_EXPIRED`, `TOKEN_INVALID`, `INSUFFICIENT_ROLE`) are consumed by the Phaser

client to show the correct UI error state.

```
import { Request, Response, NextFunction } from 'express'; import jwt from
'jsonwebtoken'; export interface AuthRequest extends Request { user?:
{   userId: string;   username: string;   role:   string;   }; } export const
requireAuth = ( req: AuthRequest, res: Response, next: NextFunction ): void
=> {   const authHeader = req.headers.authorization;   if (!
authHeader?.startsWith('Bearer ')) {     res.status(401).json({ error: 'Unauthorized',
code: 'NO_TOKEN' });     return;   }   const token = authHeader.slice(7);   try
{     const payload = jwt.verify(     token,
process.env.JWT_ACCESS_SECRET!   ) as { userId: string; username: string;
role: string };     req.user = payload;     next();   } catch (err: any) {     if (err.name
=== 'TokenExpiredError') {       res.status(401).json({ error: 'Token expired', code:
'TOKEN_EXPIRED' });     } else {       res.status(401).json({ error: 'Invalid token',
code: 'TOKEN_INVALID' });     }   } }; export const requireRole = (roles: string[]) =>
(req: AuthRequest, res: Response, next: NextFunction): void => {   if (!req.user || !
roles.includes(req.user.role)) {     res.status(403).json({ error: 'Forbidden', code:
'INSUFFICIENT_ROLE' });     return;   }   next();   };
```

7.6 Part 4A Completion Checklist

Use this checklist to verify all steps in Part 4A are complete before moving to Part 4B. Every item must be checked before proceeding. Items are grouped by phase.

► Server Bootstrap

- Server dependencies installed on Mac and committed (`package.json` + `package-lock`)
- `apps/server/package.json` scripts section complete (`dev`, `build`, `start`, `test`, `lint`)
- `tsconfig.json` created with `@shared/*` path alias and committed

- `src/redis.ts` Redis client singleton created with error/connect/reconnect handlers
- `src/prisma.ts` Prisma client singleton created with global dev guard
- `src/index.ts` server entry point created with all middleware, route mounts, health endpoint, and graceful shutdown
- `pnpm build` runs on Oracle VM with no TypeScript errors — `dist/` directory appears
- `node dist/index.js` starts successfully on Oracle VM — Redis, DB, and server all confirm connected
- `curl http://localhost:3000/health` returns `{"status":"ok", ...}` on Oracle VM

► Prisma Schema

- `npx prisma init` run on Mac — `prisma/schema.prisma` and `.env` created
- Generator (`prisma-client-js`, `fullTextSearch`) and datasource (`postgresql`) blocks configured
- User model with `Role` and `UserStatus` enums defined and all relations declared
- Avatar model defined with all 9 outfit slots (`head`, `face`, `body`, `legs`, `feet`, `back`, `hand`, `hairStyle`, `eyeStyle`)
- Item model with `ItemCategory` and `ItemRarity` enums defined
- Inventory model defined with `@@unique([userId, itemId])` constraint
- Room model defined with nullable `ownerId` and `onDelete: SetNull`
- RoomFurniture model defined with `x`, `y`, `z`, `rotation`, `layer` fields
- ChatMessage model defined with `@@index([roomId, createdAt])`
- Friend model with `FriendStatus` enum and `@@unique([requesterId, addresseeId])`
- Report model with `ReportReason` and `ReportStatus` enums and `@@index([status, createdAt])`
- InviteCode model defined with `@@index([code, isActive])`

► Database Migrations

- `npx prisma migrate deploy` run on Oracle VM — no errors
- `psql \dt` shows all **10 tables** created in `havenworld_prod`
- `npx prisma generate` run on Oracle VM — client generated with no errors

► Database Seeding

- `prisma/seed.ts` created with idempotent upsert logic and committed
- `prisma.seed` script registered in `package.json` under `"prisma"` key
- `npx prisma db seed` run on Oracle VM — **4 rooms** and **17 items** created
- `psql` query confirms rows exist in `rooms` and `items` tables

► Services and Middleware

- `src/services/InventoryService.ts` created with `grantDefaultItems`, `getUserInventory`, `hasItem`, `hasAllItems`
- `src/middleware/auth.ts` created with `requireAuth` and `requireRole` factory
- All files committed and pushed to `main` — Oracle VM confirmed in sync with `git status`

► Continuation — What Comes Next

This is **Part 4A**. The server is bootstrapped and the database is fully migrated and seeded. The next sub-parts are:

- **Part 4B** — Complete authentication route (registration, email verification, login, token refresh, logout); Socket.io real-time handlers (`auth:join`, `player:move`, `chat:send`, `avatar:update`, `disconnect`); the `RoomManager` in-memory state service; and the `ModerationService`.
- **Part 4C** — Admin and friend routes; invite code generation; the full deployment workflow (`git pull` → `pnpm install` → `pnpm build` → `npx prisma migrate deploy` → `pm2 reload`); and the final alpha-ready gate checklist.

Do not start Part 4B until all items on the Part 4A Completion Checklist above are checked.

